

Iluminação local: conceito e hipótese do modelo

Iluminação em Computação Gráfica

Em computação gráfica, **iluminar** significa calcular a **cor final visível** de um ponto da superfície

Essa cor é resultado da interação entre:

- propriedades da luz
- propriedades do material
- orientação da superfície
- posição do observador

Iluminação é parte do processo de síntese de imagens, não de modelagem geométrica

Cor do objeto × aparência visual

A cor armazenada no objeto **não é a cor final exibida**

A aparência muda conforme:

- posição da luz
- intensidade da luz
- direção de observação

O mesmo objeto pode parecer muito diferente sem alterar sua geometria



Modelo de iluminação local

Um **modelo de iluminação** define como a luz afeta a cor de um ponto da superfície

É uma **função matemática** aplicada localmente

Hipóteses do **modelo local**:

- considera **apenas a luz que chega diretamente ao ponto**
- cada ponto da superfície é tratado de **forma independente**
- **não há reflexão indireta** nem múltiplos rebotes
- apenas **informações locais** são usadas no cálculo

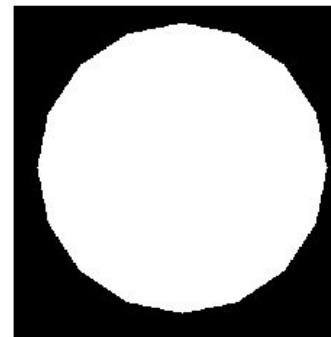
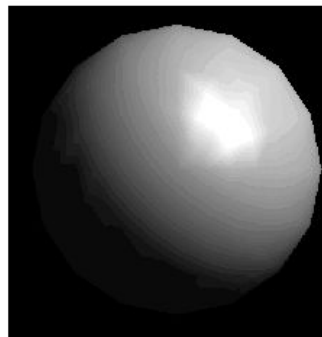
Consequências da iluminação local

Modelo computacionalmente simples

Resultados previsíveis e controláveis

Fenômenos reais ignorados:

- reflexão indireta
- oclusão
- iluminação global

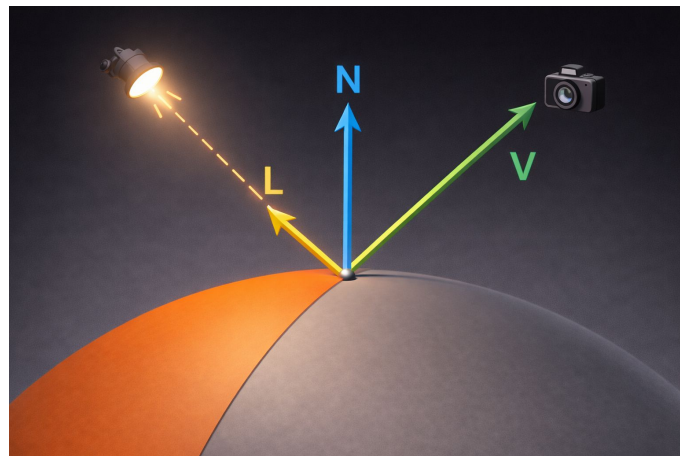


Vetores da iluminação local

O cálculo de iluminação usa três vetores principais:

- **N** - normal da superfície - define para onde a **superfície 'aponta'**
- **L** - direção da luz - define de onde a **energia chega**
- **V** - direção do observador - define de onde a superfície é vista

Todos os vetores são **unitários**, e a **intensidade** da luz depende do **ângulo entre esses vetores**



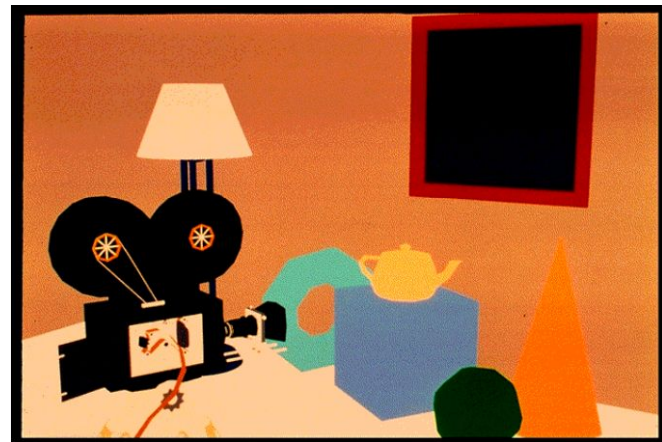
Iluminação ambiente

A iluminação local não modela reflexões indiretas.

Para **evitar regiões totalmente escuras**, usa-se o termo ambiente.

Características:

- luz **uniforme** na cena
- independe da orientação da superfície
- independe da posição da luz
- aproximação simples da iluminação indireta



Modelo matemático da iluminação ambiente

A intensidade ambiente refletida é dada por:

$$I_a = L_a * K_a$$

- **L_a**: intensidade da luz ambiente
- **K_a**: coeficiente de reflexão ambiente do material

O cálculo é feito componente a componente (RGB)

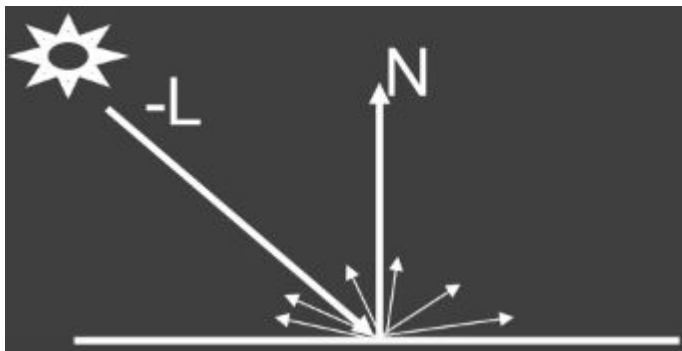
Iluminação difusa

Superfícies foscas refletem luz igualmente em todas as direções

A intensidade refletida depende apenas:

- da **direção da luz**
- da **orientação da superfície**

A **posição do observador** não influencia o resultado



Modelo matemático da iluminação difusa

A intensidade difusa refletida é dada por:

$$I_d = L_d * K_d * \max(0, N \cdot L)$$

- **L_d**: intensidade da luz difusa
- **K_d**: coeficiente de reflexão difusa do material

Reflexão Difusa



$K_d = 0.4, 0.55, 0.7, 0.85, 1.0$



$I_a = I_p = 1.0, K_d = 0.4, K_a = 0.0, .015, 0.3, 0.45, 0.6$

Propriedades da iluminação difusa

- **intensidade máxima** quando a luz **incide perpendicularmente**
- **intensidade diminui** conforme o **ângulo aumenta**
- **intensidade nula** quando a **luz é tangente**

Características:

- **independe da posição do observador**
- **define claramente a forma do objeto**

Orientação da superfície

A **quantidade de luz** refletida depende da **orientação da superfície**

Essa orientação é representada pela **normal da superfície**

Normal:

- vetor **perpendicular** à superfície no ponto
- define como a superfície está orientada em relação à luz

Produto escalar na iluminação

O alinhamento entre luz e superfície é medido pelo produto escalar:

$$\mathbf{N} \cdot \mathbf{L} = \cos(\theta)$$

θ é o ângulo entre:

- normal da superfície
- direção da luz

Observação importante:

- O produto escalar **não depende da distância**
- Atenuação por distância é um efeito separado
- Neste modelo, considera-se apenas a direção da luz

Interpretação de $N \cdot L$

$N \cdot L = 1 \rightarrow$ iluminação máxima

$N \cdot L = 0 \rightarrow$ luz tangente à superfície

$N \cdot L < 0 \rightarrow$ luz vindo por trás

O modelo usa $\max(0, N \cdot L)$ para garantir que apenas luz incidente contribua.

A iluminação difusa segue o modelo de Lambert e assume **reflexão perfeitamente difusa**, o que é fisicamente plausível para muitos materiais

Normais por face e por vértice

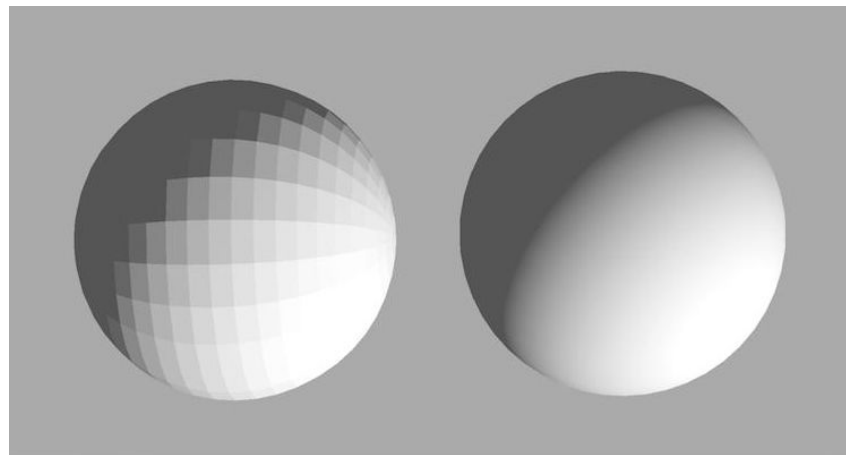
Normais por face:

- uma única normal por polígono
- aparência facetada

Normais por vértice:

- normais interpoladas
- aparência mais suave

A escolha da normal altera diretamente o resultado da iluminação



Cálculo da **normal de uma face**

A normal de um polígono pode ser obtida a partir de dois vetores da superfície. Se três vértices do triângulo são **P0**, **P1** e **P2**, definimos dois vetores do plano:

- $\mathbf{U} = \mathbf{P1} - \mathbf{P0}$
- $\mathbf{V} = \mathbf{P2} - \mathbf{P0}$

A normal é obtida pelo produto vetorial:

- $\mathbf{N} = \mathbf{U} \times \mathbf{V}$

E depois normalizamos esse vetor:

- $\mathbf{N} = \text{normalize}(\mathbf{N})$

O produto vetorial de $\mathbf{U} \times \mathbf{V}$ é:

$$\begin{pmatrix} U_y V_z - U_z V_y, \\ U_z V_x - U_x V_z, \\ U_x V_y - U_y V_x \end{pmatrix}$$

Exemplo

Dados os seguintes vértices de um triângulo:

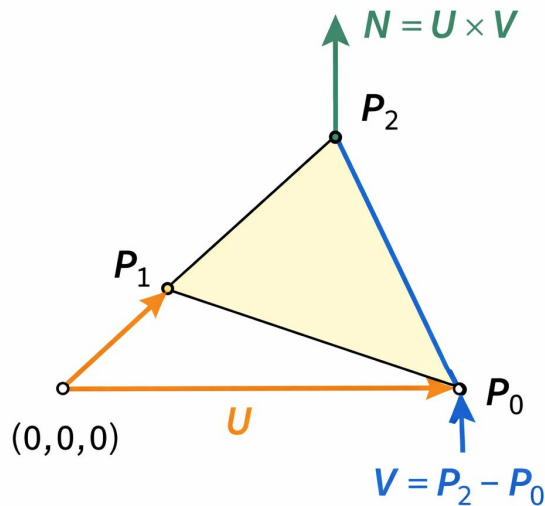
- $P_0 = (0,0,0)$
- $P_1 = (1,0,0)$
- $P_2 = (0,1,0)$

Calculamos dois vetores plano:

- $U = P_1 - P_0 = (1,0,0) - (0,0,0) = (1,0,0)$
- $V = P_2 - P_0 = (0,1,0) - (0,0,0) = (0,1,0)$

E o produto vetorial:

- $N = U \times V = (0,0,1) \Rightarrow$ já está **normalizado**



Cálculo da **normal por vértice**

Uma normal por vértice é obtida a partir das normais das faces adjacentes.

Passos:

- calcular a normal de cada face vizinha
- somar essas normais
- normalizar o resultado

Fórmula

Se um vértice pertence a várias faces com as seguintes normais:

- N_1
- N_2
- N_3
- ...
- N_k

Em muitos sistemas usa-se média ponderada pela área da face, para evitar que triângulos muito pequenos distorçam a normal:

$$N_v = \text{normalize}(A_1 N_1 + A_2 N_2 + A_3 N_3 + \dots)$$

A normal do vértice é:

- $N_v = \text{normalize}(N_1 + N_2 + N_3 + \dots + N_k)$



Exemplo

Suponha três faces ao redor do vértice:

- $N1 = (0,0,1)$
- $N2 = (0,1,1)$
- $N3 = (1,0,1)$

Somamos:

- $N = N1 + N2 + N3$
- $N = (0,0,1) + (0,1,1) + (1,0,1)$
- $N = (1,1,3)$

Depois normalizamos:

- $Nv = \text{normalize}(1,1,3)$

Especular - Superfícies não difusas

Superfícies polidas refletem luz de forma direcional

A intensidade refletida varia com a **posição do observador**

Esse efeito é chamado de **reflexão especular**

Produz regiões de brilho concentrado (**highlights**)

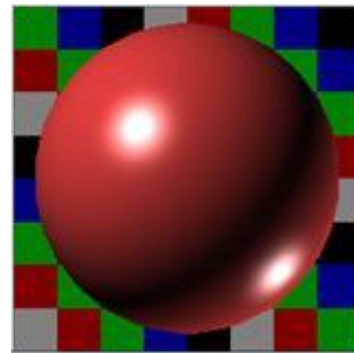
Dependência do observador

Diferente da iluminação difusa, a especular depende de:

- direção da luz
- orientação da superfície
- direção de observação

O brilho se desloca quando o observador se move

Esse efeito reforça a percepção de material



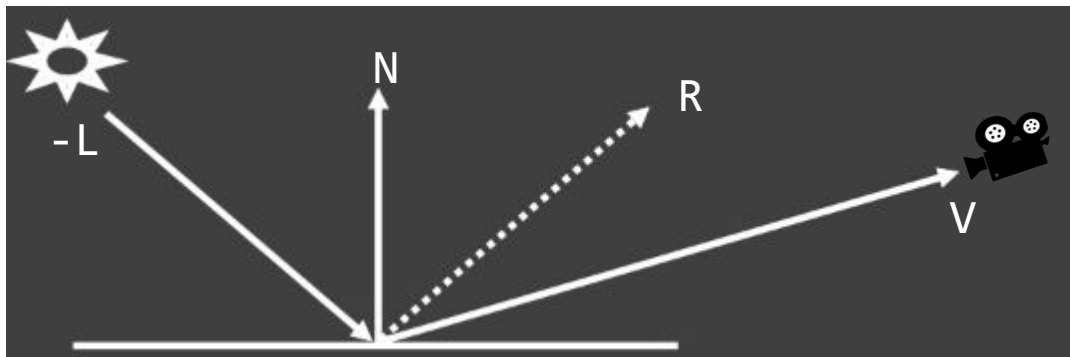
Vetores envolvidos na reflexão especular

N: normal da superfície

L: direção da luz

V: direção do observador

R: direção da luz refletida em relação à normal $\Rightarrow \mathbf{R} = -\mathbf{L} + 2(\mathbf{L} \cdot \mathbf{N})\mathbf{N}$



Termo especular do modelo de Phong

A intensidade especular depende do alinhamento entre **R** (vetor refletido) e **V** (direção do observador), medido por **R • V**. Quanto maior o alinhamento, maior o brilho especular.

O modelo de Phong define a intensidade especular como:

$$I_s = L_s * K_s * \max(0, R \cdot V)^n$$

- **L_s**: intensidade da luz especular
- **K_s**: coeficiente de reflexão especular
- **n**: expoente especular do material
- **R = -L + 2 * (L • N) * N**

Propriedades

O expoente especular controla a **concentração do brilho**

- Valores baixos → brilho largo, mas aparência mais fosca
- Valores altos → brilho concentrado, mas aparência mais polida

A cor do **brilho especular** depende da **cor da luz**

Não depende da cor difusa do objeto

Superfícies coloridas apresentam **highlights da cor da luz**

Limitações do uso do vetor refletido

O cálculo do **vetor R** pode ser numericamente instável

Pequenas variações em N podem gerar grandes variações em R

O custo computacional é maior que alternativas equivalentes

Motivação para o modelo de Blinn-Phong

Busca **simplificar o cálculo especular**

Mantém **aparência visual semelhante** ao Phong clássico

Mais estável numericamente

Mais adequado para implementação em shaders

Half-vector (vetor intermediário)

Define-se o vetor H como:

$$H = \text{normalize}(L + V)$$

H representa a direção intermediária entre luz e observador, e evita o cálculo explícito do vetor refletido

Termo especular no modelo de Blinn-Phong

A intensidade especular passa a ser calculada como:

$$I_s = L_s * K_s * \max(0, N \cdot H)^n$$

- substitui $R \cdot V$ por $N \cdot H$
- O expoente n mantém papel semelhante

Comparação conceitual: Phong × Blinn-Phong

Phong usa vetor refletido, sendo mais intuitivo geometricamente.

Blinn-Phong usa half-vector, o que é mais eficiente computacionalmente

Ambos são **modelos locais empíricos**

Blinn-Phong é:

- Amplamente utilizado em **engines clássicas**
- **Base** de muitos exemplos de **iluminação em tempo real**
- Frequentemente o primeiro modelo implementado em shaders
- Serve como referência para modelos mais avançados

Atenuação

Se projetarmos duas superfícies paralelas com o mesmo material, a equação básica não considera a **distância até a fonte de luz**.

Para incorporar esse efeito, utiliza-se um fator de atenuação *fatt*, aplicado às componentes **difusa** e **especular**:

$$F_{att} = \frac{1.0}{K_c + K_l * d + K_q * d^2}$$

Esse modelo reduz a intensidade da luz com o aumento da distância *d*: quanto mais distante a fonte, menor a iluminação percebida.

Kc, *Kl* e *Kq* são constantes definidas pelo usuário que controlam o comportamento da atenuação.

Interpretação do modelo de Phong

O modelo de Phong:

- É um modelo empírico, não fisicamente exato
- Fácil de controlar artisticamente
- Produz resultados visuais consistentes
- Amplamente usado em gráficos em tempo real

Modelo de iluminação local completo

A cor final em um ponto é a soma das contribuições:

- ambiente
- difusa
- especular

Cada termo modela um aspecto distinto da interação luz-superfície

A iluminação final é:

$$I_f = I_a + I_d + I_s$$

- **I_a** - ambiente
- **I_d** - difusa
- **I_s** - especular

Iluminação e sombreamento não são a mesma coisa

Iluminação define o **modelo matemático** da interação luz–superfície

Sombreamento define **onde e com que frequência** esse modelo é avaliado

O mesmo modelo de iluminação pode gerar resultados visuais diferentes

A diferença está no nível geométrico do cálculo

Sombreamento **Flat**

O cálculo de iluminação é feito uma **única vez por face**

Usa uma única normal por polígono

Resultado visual:

- superfícies facetadas
- transições abruptas de iluminação

Computacionalmente simples

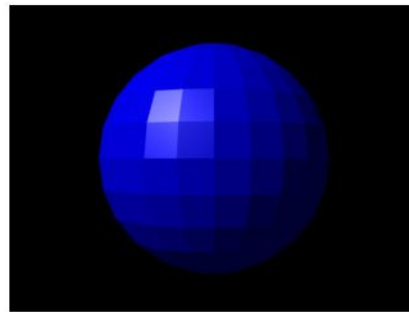
Características do Flat Shading

Não interpola normais

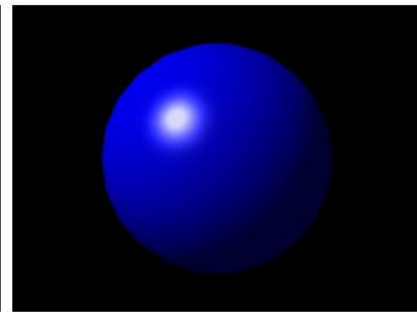
Não interpola intensidades

Evidencia a estrutura poligonal do modelo

Útil para depuração geométrica



FLAT SHADING



PHONG SHADING

Sombreamento Gouraud

A iluminação é calculada **nos vértices**

As intensidades são interpoladas ao longo da face

Requer normais por vértice

Produz superfícies visualmente mais suaves

Limitações:

- Destaques especulares podem desaparecer
- O brilho depende da posição dos vértices
- Não captura bem variações rápidas de iluminação
- Problema evidente em objetos com poucos polígonos

Sombreamento Phong

As normais são **interpoladas** ao longo da face

A iluminação é calculada **por fragmento**

Produz resultados visualmente mais realistas

Custo computacional maior

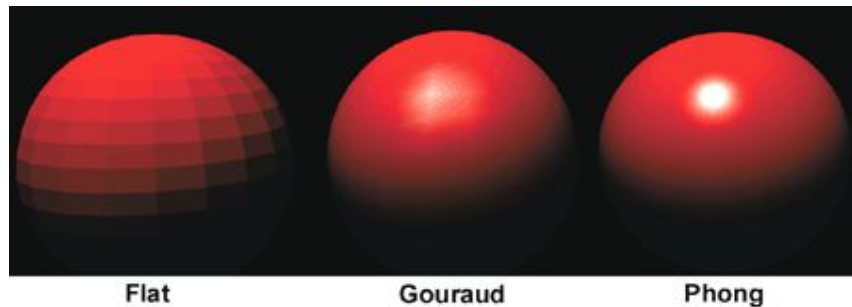
Comparação conceitual dos modelos de sombreamento

Flat: iluminação por face

Gouraud: iluminação por vértice

Phong: iluminação por fragmento

O modelo de iluminação é o mesmo, muda o local do cálculo



Relação com hardware gráfico moderno

GPUs modernas favorecem **cálculo por fragmento**

O **custo** de Phong shading é **aceitável**

A **iluminação local** é normalmente avaliada no **fragment shader**

Flat e Gouraud são hoje escolhas conceituais

Onde a iluminação acontece hoje

Em OpenGL moderno, a iluminação é calculada na GPU, especificamente no **fragment shader**

O modelo local é avaliado **por fragmento**

A CPU não calcula iluminação ponto a ponto

A GPU executa o modelo em paralelo para cada fragmento

Dados usados no fragment shader

Entrada:

- normal da superfície (N)
- direção da luz (L)
- direção do observador (V)

Parâmetros adicionais:

- intensidades da luz
- coeficientes do material
- expoente especular

Fluxo do cálculo de iluminação

Geralmente segue-se o seguinte fluxo:

1. normalizar vetores
2. calcular termo ambiente
3. calcular termo difuso ($N \cdot L$)
4. calcular termo especular
5. somar contribuições

Limitações mantidas na implementação

O shader não considera:

- sombras reais
- reflexão indireta
- oclusão

Essas limitações são do modelo, não da implementação

O shader executa exatamente o que o modelo descreve

Importância da iluminação local

Base conceitual para todos os modelos posteriores

Fundamenta a relação entre:

- geometria
- matemática
- aparência visual

Mesmo modelos globais reutilizam esses termos localmente